

Experiment 1: Installation of Hypervisors and Initiation of VMs with Image File

Aim

To install and configure an open-source hypervisor and create/instantiate a Virtual Machine (VM) using an existing image file, and to demonstrate the successful installation and execution of the VM.

Tasks

- Install an open-source hypervisor such as Oracle VirtualBox.
- Obtain a suitable VM image/ISO file.
- Create a new VM using the hypervisor.
- Configure the VM's CPU, RAM, storage, and network.
- Attach the image/ISO file to the VM.
- Start and boot the VM.
- Complete the guest OS installation, if required.
- Verify that the VM is running successfully.
- Install the required build tools and Linux kernel headers.
- Record the configuration and capture relevant screenshots.

Step 1: Install the Open-Source Hypervisor (Oracle VirtualBox)

Oracle VirtualBox, a free and open-source hypervisor, was downloaded from the official VirtualBox website (virtualbox.org). The VirtualBox 7.2 platform package for Windows hosts was selected and downloaded, and installed on the host machine along with the Extension Pack.

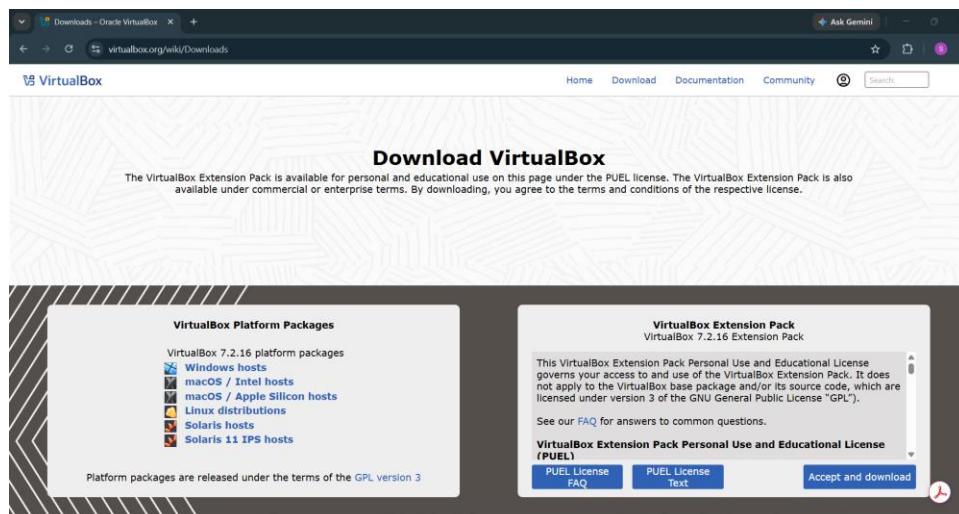


Fig 1. Official VirtualBox downloads page (virtualbox.org) used to obtain the VirtualBox 7.2.16 installer.

Step 2: Obtain the VM Image (ISO) and Create a New Virtual Machine

An Ubuntu Desktop ISO image was downloaded to be used as the installation media for the guest operating system. After installing VirtualBox, the Oracle VirtualBox Manager was opened, which presents options to create a new VM, open a saved VM, or import a VM. The 'Create a new virtual machine (VM)' option was selected to begin creating the VM.

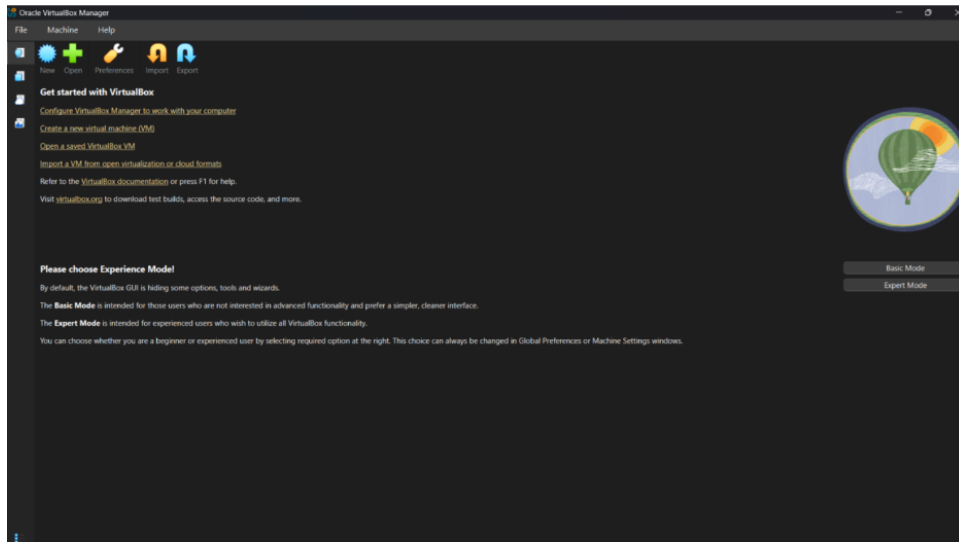


Fig 2. Oracle VirtualBox Manager home screen, showing the option to create a new virtual machine.

Step 3: Configure the VM's CPU, RAM, and Storage

In the New Virtual Machine wizard, the virtual hardware was specified: a Base Memory of 4096 MB (4 GB) of RAM, 2 virtual CPUs, and a virtual Disk Size of 25.00 GB were allocated to the VM.

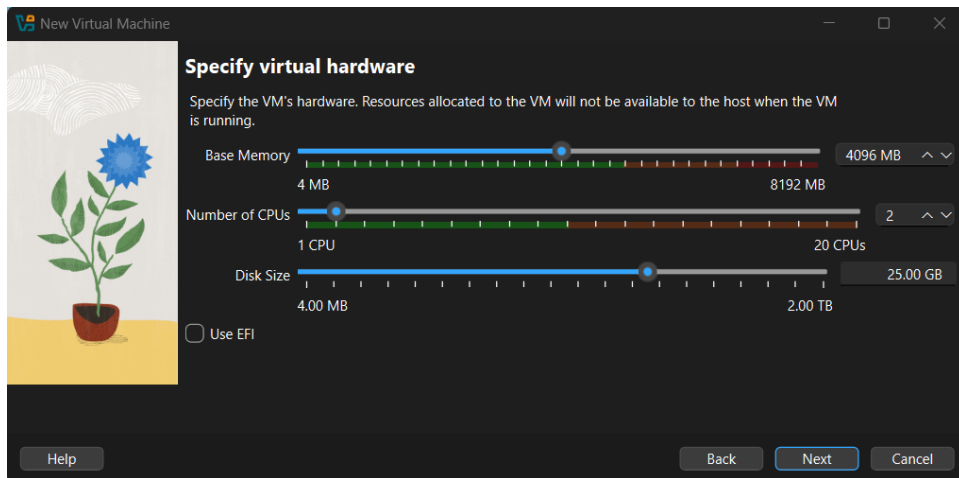


Fig 3. Specifying virtual hardware — 4096 MB base memory, 2 CPUs, and a 25 GB virtual disk.

Step 4: Attach the ISO Image File to the VM

The downloaded Ubuntu ISO image was attached as the boot/installation medium for the new VM. The wizard's summary screen confirms the final configuration before creation: VM name 'Lab 1', guest OS type 'Ubuntu (64-bit)', ISO image path set to the downloaded ubuntu-26.04-desktop-amd64.iso file, 4096 MB base memory, 2 processors, and a 25 GB hard disk. Clicking 'Finish' created the VM with this configuration.

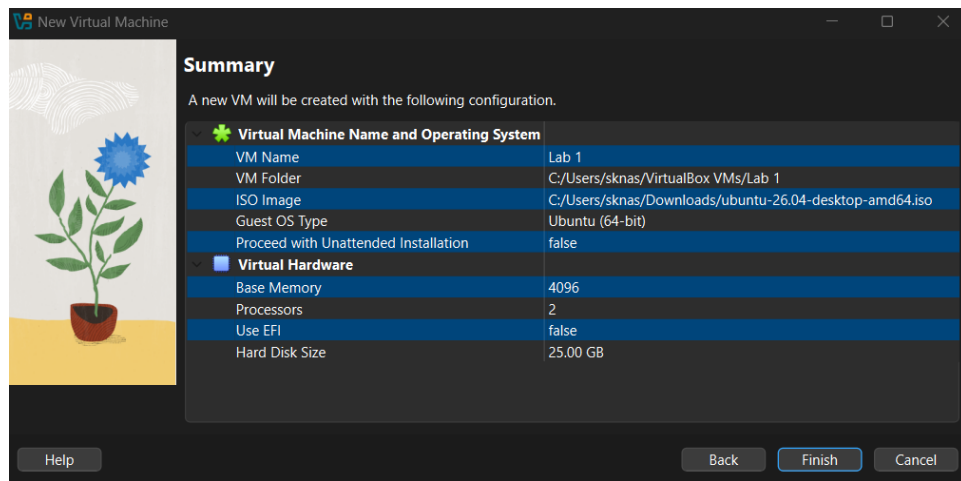


Fig 4. Summary screen confirming VM name, attached ISO image, and allocated hardware resources before creation.

Step 5: Start the VM and Complete the Guest OS Installation

The newly created VM was started from the VirtualBox Manager, and it booted from the attached ISO to begin the Ubuntu guest operating system installation. Upon completion, the installer displayed the 'Installation complete' screen confirming that Ubuntu was installed and ready to use, with the option to restart into the freshly installed system.

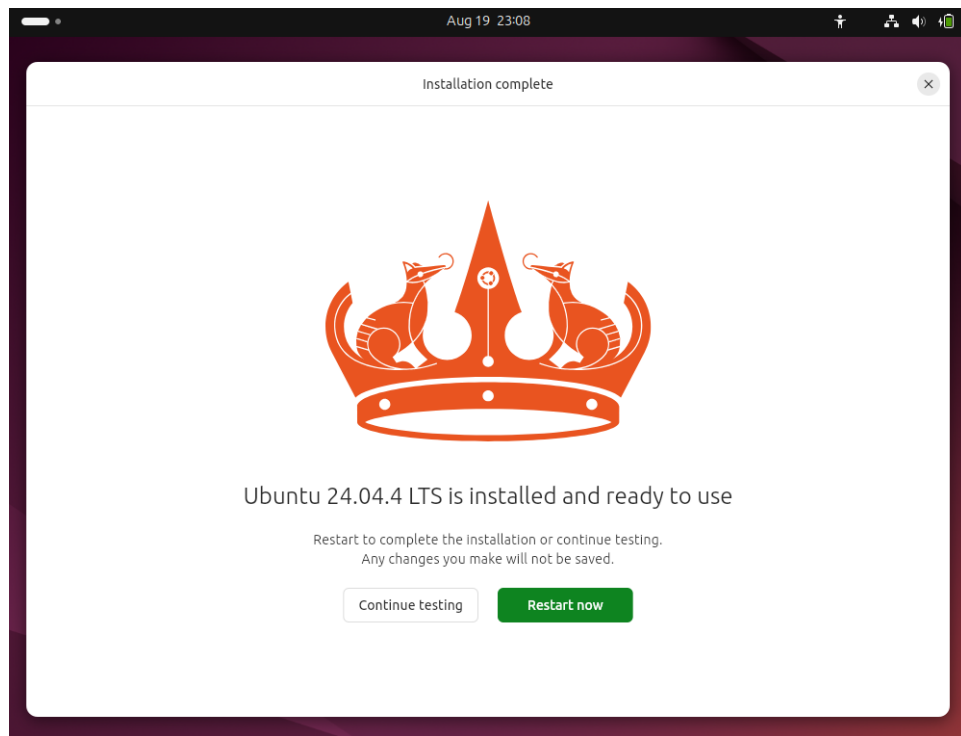


Fig 5. Guest OS installation completed — Ubuntu installed and ready to use inside the VM.

Step 6: Boot the Installed VM

After restarting, the VM booted from the newly installed disk into the Ubuntu login screen, confirming that the virtual machine powers on and boots the installed guest operating system correctly.

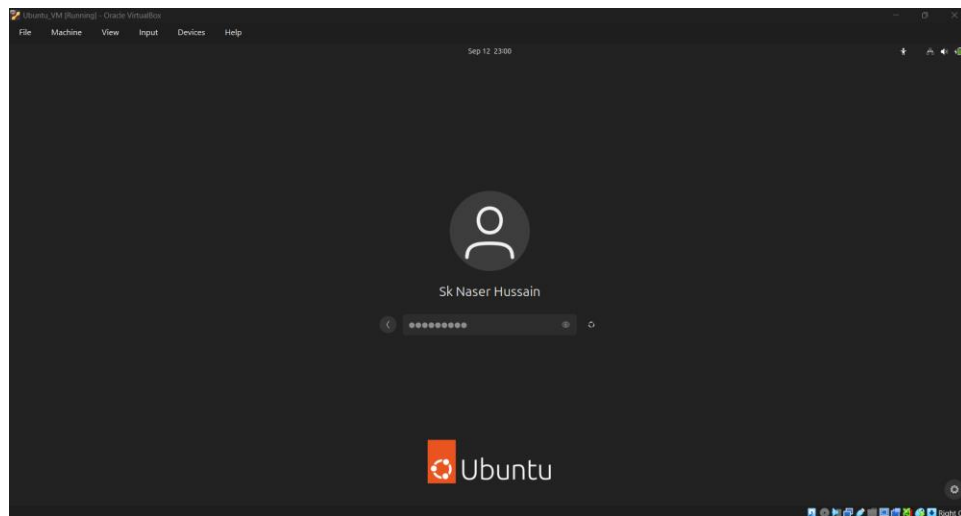


Fig 6. Ubuntu VM booted successfully into the login screen after installation, inside the running VirtualBox window.

Step 7: Verify That the VM Is Running Successfully

After logging in, the Ubuntu desktop environment loaded fully inside the VirtualBox window, with the taskbar (Firefox, Files, App Center, Help, Trash) and desktop icons visible and responsive. This confirms that the VM is running successfully with a fully functional guest operating system.

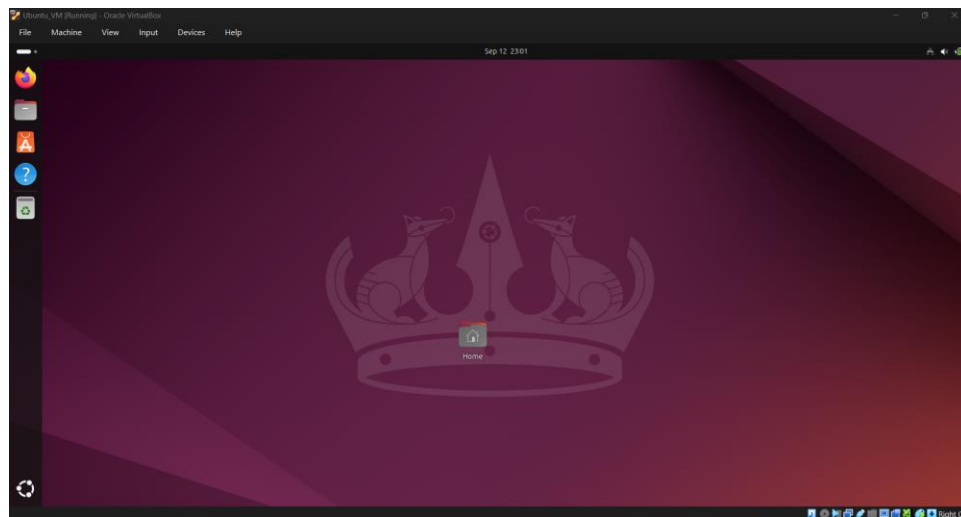


Fig 7. Ubuntu desktop running successfully inside the VM, confirming a working installation.

Step 8: Install Required Build Tools and Linux Kernel Headers

To prepare the VM for further experiments (such as building Guest Additions or kernel modules), the package index was updated and the essential build tools and the Linux kernel headers matching the running kernel were installed using the following commands in the guest OS terminal:

```
sudo apt update
sudo apt install -y build-essential linux-headers-$(uname -r)
```

These commands update the local package lists and then install the build-essential package (compilers and related tools) along with the kernel headers corresponding to the currently running kernel version, which are required for compiling kernel modules such as the VirtualBox Guest Additions.

Result / Conclusion

Oracle VirtualBox was successfully installed as the hypervisor on the host machine. A new virtual machine, 'Lab 1', was created and configured with 4096 MB of RAM, 2 virtual CPUs, and a 25 GB virtual disk, with an Ubuntu ISO image attached as the installation medium. The VM was started, the Ubuntu guest operating system installation was completed successfully, and the VM was verified to be running correctly with a fully functional desktop environment. The required build tools and Linux kernel headers were subsequently installed on the guest OS. Screenshots were captured at each stage, confirming that all the stated tasks of the experiment were achieved.